



On étudie deux types de structures de données, que l'on retrouve classiquement dans certaines situations.



Les contextes d'exécution des fonctions utilisées dans un programme sont empilés puis dépilés au gré des appels dans la **pile** (*stack*) d'appels.

Un traitement de texte mémorise les actions effectuées afin de pouvoir revenir en arrière : les actions sont empilées, puis éventuellement dépilées sur demande utilisateur (CTL+z).

De même un navigateur mémorise les pages visitées et pour pouvoir y revenir dans l'ordre inverse des visites : les pages sont empilées puis dépilées.



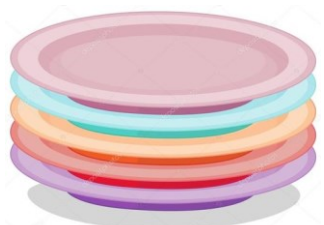
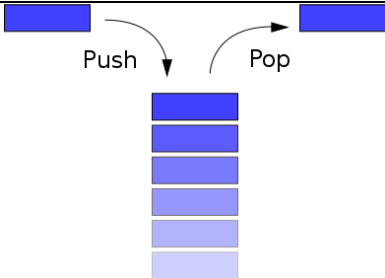
Les travaux envoyés à une imprimante sont stockés dans une file (*queue*)

La gestion des processus par le système d'exploitation est elle aussi confiée à une file.

I. LES PILES

1. L'INTERFACE

une pile stocke les données de la même manière qu'on stockerait une pile d'assiettes (ou de crêpes, ou de livres, ...) dans un tiroir étroit: on n'a pas accès aux côtés de la pile. On n'en voit que la partie haute.

	
Comment accéder à la pile d'assiettes ? - Empiler une assiette sur le haut - Enlever l'assiette qui est tout en haut sur la pile - Regarder si la pile est vide (Regarder l'assiette qui est en haut de la pile)	Interface Utilisateur d'une pile : - Empiler une valeur v en haut de la pile P: empiler(P, v) (Push) - Retirer la valeur stockée tout en haut de la pile P: depiler(P) (Pop) → renvoie la valeur dépilée est_vide(P) → renvoie un booléen sommet(P) → renvoie la valeur v au sommet de la pile P

Rq1. Il existe une opération supplémentaire nécessaire pour créer la pile (vide, à sa création) : **créer_pile() → renvoie l'identifiant de la pile créée**

Rq2. Il n'y a donc pas d'opération élémentaire pour :

- accéder à une donnée qui n'est pas sur le haut de la pile,
- savoir combien de données sont stockées dans la pile.

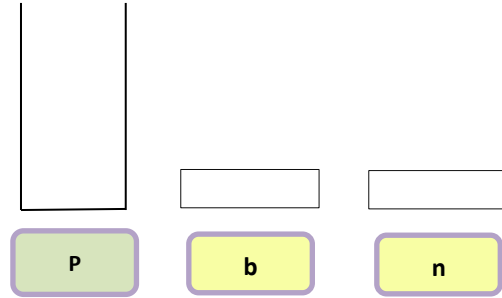
La structure de donnée Pile, munie des cinq fonctions primitives précédentes (y compris la primitive de création de la pile) définit **un type abstrait** de données.

La donnée qui sera **dépilée** par un appel à depiler() sera **toujours la dernière donnée qui a été empilée**.

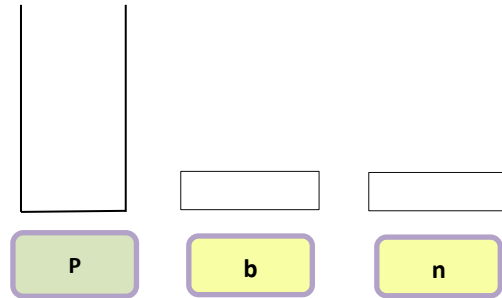
L'accès à la pile est donc en mode "**Last in, First Out**" (**LIFO**) : dernier arrivé, premier reparti.

Exo 1. ← Compléter l'état des variables P, b et n à chaque stade de l'exécution du code proposé :

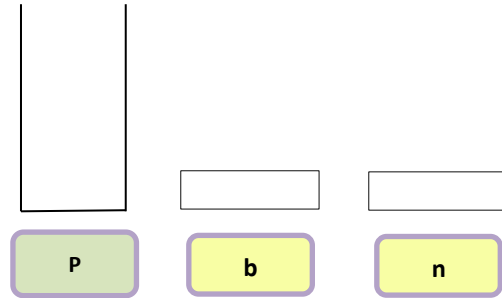
```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



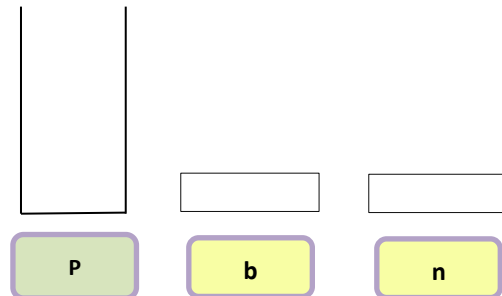
```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



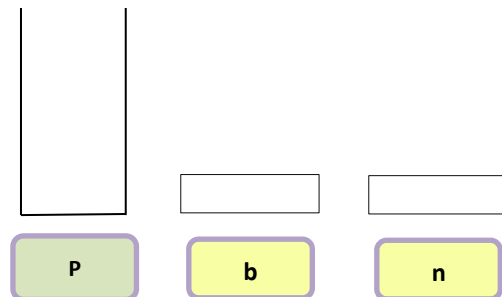
```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



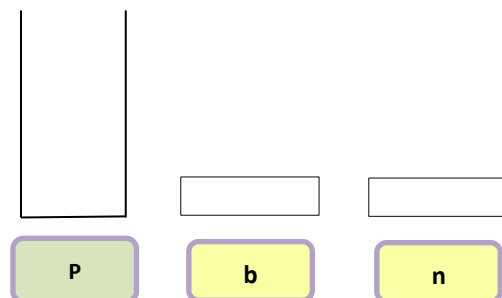
```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



```
P ← créer_pile()
b ← est_vide(P)
empiler(P, 5)
empiler(P, 8)
n = sommet(P)
empiler(P, -2)
n = sommet(P)
n = depiler(P)
b ← est_vide(P)
n = depiler(P)
```



Exo 2. On considère un navigateur web pour lequel on s'intéresse à deux opérations :

- aller à une nouvelle page
- revenir à la page précédente

a. Compléter le code ci-dessous :

```
adresse_courante = ""
adresses_precedentes = creer_pile()

def aller_a(adresse_cible):
    """visite une nouvelle page """

def retour():
    """retourne à la page precedente"""
```

- b. la fonction **retour()** n'implémente qu'un retour en arrière.
Comment pourrait-on implémenter une fonction **retour_en_avant ()** ?



SY Piles et Files - Exo Navigateur.docx

Rq 3. En lieu et place des fonctions ordinaires **est_vide(p)**, **empiler(p,v)** et **depiler(p)** définissant l'interface, l'utilisation des classes permettrait de même de définir une classe pile définissant les méthodes **est_vide()**, **empiler(v)** et **depiler()**

L'ajout d'un élément **v** dans la pile **p** devient alors **p.empiler(v)**

2. IMPLEMENTATION EN PYTHON



La classe 'list' en Python définit deux méthodes qui permettent d'implémenter une pile :

- **append(el)**: ajoute l'élément en fin de liste.
- **pop()**: supprime le dernier élément de la liste et le renvoie.

```
>>> stack = [3, 4, 5]
>>> type(stack)
<class 'list'>
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```



<https://docs.python.org/3/tutorial/datastructures.html#using-lists-as-stacks>

Rq4. En python ces opérations `append()` et `pop()` s'exécutent en moyenne en temps constant (coût en $O(1)$). C'est l'implémentation des listes en Python qui le permet mais ce n'est pas le cas dans tous les langages.

Exo 3. Création d'une classe pile avec une liste chaînée :

La structure de liste chaînée permet d'implémenter facilement la structure de pile :

empiler un élément revient à ajouter un maillon en tête de liste et dépiler revient à supprimer le maillon de tête.

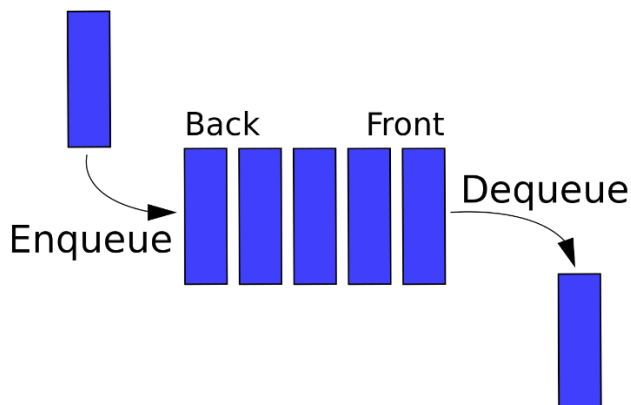
On importe la classe maillon.

Compléter le fichier [une pile avec une liste chaînée TBC.py](#)

II. LES FILES

1. L'INTERFACE

Les files (*queues* en anglais) correspondent également à la notion de file d'attente dans la vie courante: (une file d'attente à la caisse, à un feu rouge...):



Lorsqu'on ajoute un élément, celui-ci se retrouve à la fin de la queue, et on retire les éléments dans l'ordre dans lequel ils sont arrivés, suivant le principe du "**First in, First Out**" (**FIFO**) : premier arrivé, premier servi.

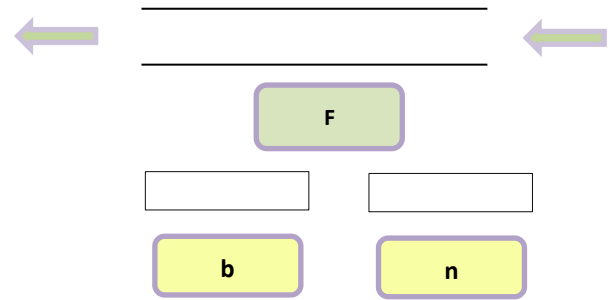
Interface Utilisateur d'une file :

- créer une file vide :
`créer_file()` → renvoie l'identifiant de la file créée
- ajouter une valeur v dans la file F :
`ajouter(F, v)` (*Push*)
- Retirer la valeur à l'avant de la file:
`retirer(F)` (*Pop*) → renvoie la valeur retirée
- `est_vide(F)` → renvoie un booléen
- `premier(F)` → renvoie la valeur v à l'avant de la file F

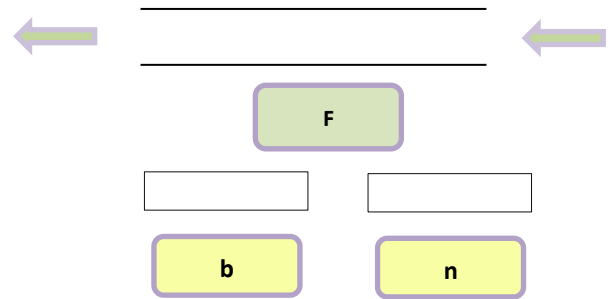
La structure de donnée File, munie des cinq fonctions primitives précédentes définit un nouveau **type abstrait** de données.

Exo 4. ← Compléter l'état des variables **f**, **b** et **n** à chaque stade de l'exécution du code proposé :

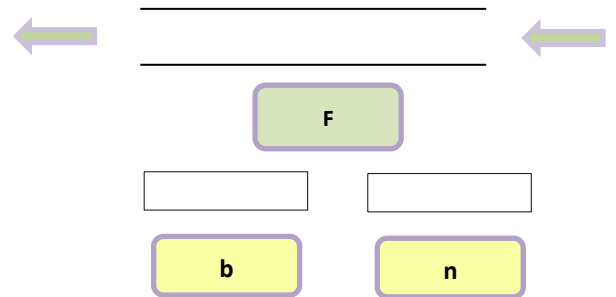
```
F←créer_file()
b=est_vide(F)
ajouter(F, 5)
ajouter (F, 8)
ajouter (F, -2)
n←retirer(F)
n←premier(F)
b←estVide(F)
```



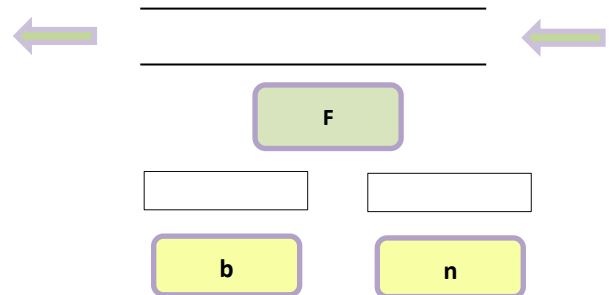
```
F←créer_file()
b=est_vide(F)
ajouter(F, 5)
ajouter (F, 8)
ajouter (F, -2)
n←retirer(F)
n←premier(F)
b←estVide(F)
```



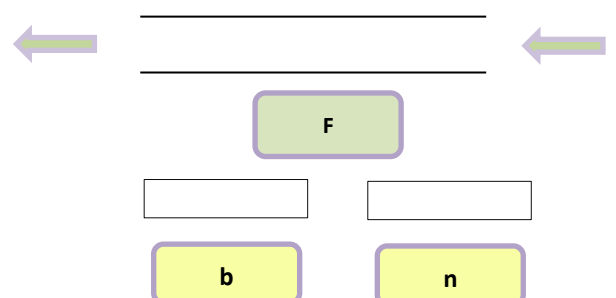
```
F←créer_file()
b=est_vide(F)
ajouter(F, 5)
ajouter (F, 8)
ajouter (F, -2)
n←retirer(F)
n←premier(F)
b←estVide(F)
```



```
F←créer_file()
b=est_vide(F)
ajouter(F, 5)
ajouter (F, 8)
ajouter (F, -2)
n←retirer(F)
n←premier(F)
b←estVide(F)
```



```
F←créer_file()
b=est_vide(F)
ajouter(F, 5)
ajouter (F, 8)
ajouter (F, -2)
n←retirer(F)
n←premier(F)
b←estVide(F)
```



2. IMPLEMENTATION EN PYTHON



La classe `list` en Python définit deux méthodes qui permettent d'implémenter une file :

- **list.append(e1)**: ajoute l'élément en fin de liste. (implémente la fonction ajouter(list)
- **list.pop(0)**: supprime le premier élément de la liste et le renvoie. (implémente la fonction retirer())



Cependant, les listes ne sont pas très efficaces pour réaliser ce type de traitement. Alors que les ajouts et suppressions en fin de liste sont rapides, les opérations d'insertions ou de retraits en début de liste sont lentes (car tous les autres éléments doivent être décalés d'une position).



Pour implémenter une file, utilisez la classe `collections.deque` qui a été conçue pour réaliser rapidement les opérations d'ajouts et de retraits aux deux extrémités.

Ex 1 :

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")          # Terry arrives
>>> queue.append("Graham")        # Graham arrives
>>> queue.popleft()               # The first to arrive now leaves
'Eric'
>>> queue.popleft()               # The second to arrive now leaves
'John'
>>> queue                         # Remaining queue in order of arrival
deque(['Michael', 'Terry', 'Graham'])
```



<https://docs.python.org/3/tutorial/datastructures.html#using-lists-as-queues>

Exo 5. Création d'une classe file avec une liste chaînée :

La structure de liste chaînée permet d'implémenter facilement la structure de file :

retirer un élément revient à supprimer le maillon de tête et ajouter un élément revient à ajouter un maillon en queue de liste.

Dans la structure de liste, ajouter un élément en queue oblige alors à parcourir toute la liste jusqu'à trouver le dernier maillon.

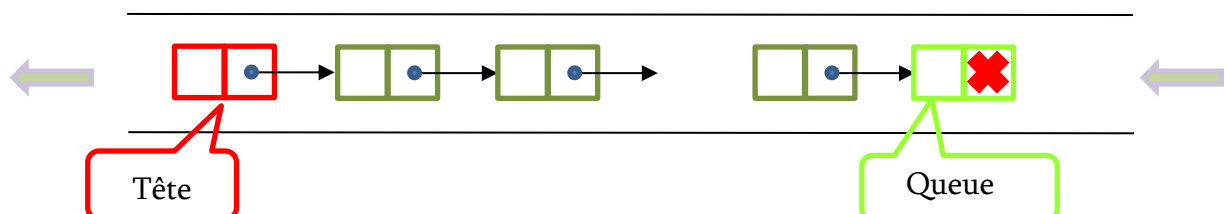
Il apparaît alors intéressant d'enrichir la structure d'un attribut permettant d'accéder directement au dernier maillon de la chaîne.

```
from classe_maillon import *

class File:
    """structure de file"""
    def __init__(self):
        self.tete = None #maillon de tete
        self.queue = None #maillon de queue

    def est_vide(self):
        return self.tete is None
```

Compléter le fichier [une file avec une liste chaînée TBC.py](#) qui importe [classe_maillon.py](#)



III. Exos

Exo 6. Quel est l'état des variables à l'issue de l'exécution de ce programme :

```
P ← créer_pile()
empiler(P, -6)
empiler(P, 3)
n ← sommet(P)
depiler(P)
empiler(P, 7)
depiler(P)
n ← sommet(P)
b ← estVide(P)
```

Exo 7. Quel est l'état des variables à l'issue de l'exécution de ce programme :

```
F ← NewFile()
ajouter(F, -6)
ajouter (F, 3)
n ← valeurDebut(F)
n ← retirer(F)
ajouter (F, 7)
n ← retirer(F)
n ← valeurDebut(F)
b ← estVide(F)
```

Exo 8. Quelle(s) structure(s) de données choisir (liste(s), dictionnaire(s), pile(s), file(s)) pour chacune de ces modélisations :

- L'historique des actions d'un traitement de texte exploitable avec une commande **annuler** (*undo*)
- La playlist de la soirée.
- Un jeu de bataille entre Alice et Bob.
- Un répertoire téléphonique.

Exo 9. Classe Pile

- Compléter la classe Pile avec les 3 méthodes **sommet()**, **vider()** et **taille()**.
- Indiquer pour chacune des méthodes précédentes le nombre d'opérations effectuées (le nombre d'accès à la liste mesure la complexité de la méthode), c'est-à-dire sa complexité en fonction de la taille n du nombre d'éléments.
sommet() :
vider() :
taille() :
- Modifier la classe Pile en lui ajoutant un attribut **_taille** qui permettra d'améliorer la complexité de la méthode **taille()**.
- Compléter la classe Pile avec une fonction **inverser_pile(p)** qui prend en argument une pile p et qui renvoie une autre pile avec les éléments empilés dans l'ordre inverse.

Exo 10. Deux piles pour une file.

- Ecrire une classe **File** dont les deux attributs sont des instances d'une classe Pile.
- Définir une méthode **ajouter(v)** qui ajoute un élément v dans la file.
- Définir une méthode **retirer()** qui retire un élément de la file et le renvoie.



Il faut voir la file comme une juxtaposition de deux piles avant et arrière dont les fonds coïncident

Exo 11. Permute le fond et le sommet.

def haut_bas_bas_haut(p):

Inverse uniquement le sommet et le bas de la pile p. Modifie p, sans retour.